Recommendation Systems for Exploratory Data Tasks [Vision]

Anna Fariha
University of Utah
Salt Lake City, Utah, USA
afariha@cs.utah.edu

Whanhee Cho
University of Utah
Salt Lake City, Utah, USA
whanhee.cho@utah.edu

ABSTRACT

Exploratory Data Tasks (EDTs) involve iteratively steering workflows as users refine their subjective goals based on newly emerging insights. EDTs are performed by millions of users with varying levels of expertise to understand unfamiliar data, discover trends, and identify evidence that informs critical decision-making. However, a key challenge in EDTs is the enormous space of possible *actions* that one can take at each step: users struggle to choose among thousands of joins, transformations, and aggregations, leading to "exploration paralysis". Because EDT workflows are interconnected, each choice shapes subsequent exploration, and suboptimal choices can lead to inefficiency, missed insights, confirmation bias, and incomplete coverage. This calls for intelligent recommendations that efficiently guide users toward optimal EDT actions.

We envision *recommendation as a core capability of data systems*, proactively guiding users toward promising actions that lower the barrier to EDTs. This capability becomes even more critical as AI agents emerge as a new class of users that perform EDTs on behalf of humans. EDT recommendation is challenging because the action space is combinatorial and actions are data-dependent, which require costly materialization. Moreover, recommendation often involves bundles or sequences of actions across interdependent tasks, requiring coordination across tasks. In this paper, we present our vision of EDT recommendation systems along two axes: single-task vs. multi-task settings and single-action vs. multi-action recommendations. We outline a research agenda that progresses from recommending individual EDT actions to constrained bundles and sequences of actions, and ultimately to coordinated recommendations across interconnected EDTs. We identify research directions for incorporating various contexts (user, data, task, and ecosystem), addressing efficiency challenges, and coordinating across tasks.

PVLDB Reference Format:
Anna Fariha and Whanhee Cho. Recommendation Systems for Exploratory Data Tasks. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

In today's world, users are overwhelmed by abundant content (e.g., products, movies, news), making recommendation essential for information filtering. Recommendation systems assist users in discovering relevant items from a large pool of candidates—by analyzing past behavior, preferences, and contextual signals—while striking a balance between exploration (discovery) and exploitation (personalization). Beyond filtering content, recommendation can also guide users through complex tasks by suggesting next actions. Such task recommendations support rapid onboarding for novices and improve efficiency for experts. While recommendation systems are well-studied in content domains, their use in data management tasks remains underdeveloped.

We focus on *Exploratory Data Tasks (EDTs)* [4], where users iteratively steer workflows, refining subjective goals as new insights emerge. For instance, data analysts choose which data subsets to drill into based on emerging patterns, and database administrators reason about diagnostic queries and logs to guide subsequent debugging steps. Similar examples arise across numerous applications such as exploratory data analysis [42], summarization [17], insight discovery [88], explanation [80], and visualization [85]. As data becomes central to decision-making, EDTs are ubiquitous across users of all expertise levels. Moreover, the users of data systems now include AI agents that perform EDTs on behalf of humans, taking exploratory actions at machine speed. However, users must repeatedly choose the next action from thousands of options such as joins, aggregations, transformations, etc. Suboptimal choices lead to (1) *inefficiency*, wasting human time, effort, and computational resources on futile directions; (2) *confirmation bias*, arising from an unprincipled exploration process; and (3) *missed insights*, due to portions of the data remaining unexplored, which may lead to incorrect conclusions and adversely affect critical decision-making. These issues are amplified for agents: without guidance, agents explore by generating and executing actions through trial and error, which significantly increases the load on the data systems.

Exploration is an iterative process and choosing the next best action at each step is challenging due to two key reasons. First, unlike fully observable entities such as tuples in a database, EDT actions are merely *programs parameterized by the underlying data*. Thus, the utility of an action is difficult to assess without materializing it over the data. Second, the space of possible EDT actions is often *combinatorial* (Example 1.1), which overwhelms users with too many choices, leading to "exploration paralysis". Programmatically enumerating and materializing this vast space of candidate actions is computationally prohibitive, motivating the need for *recommendations* toward choosing optimal EDT actions.

Example 1.1 (Combinatorial search space of EDT actions). Consider a business dataset with the exploratory goal of identifying discrepancies across groups under some (unknown) data partitioning, by computing various aggregated attributes such as `AVG(salary)` or `SUM(cost)` grouped by various attribute combinations such as `gender & education` or `region & year`. Even with only 5 aggregate functions over 20 numeric attributes and 20 grouping attributes, and restricting queries (actions) to only 2 aggregate and 2 grouping attributes, the number of possible choices is $5^2 \cdot \binom{20}{2} \cdot \binom{20}{2} \approx 1$ million!

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

User perception of exploratory findings is highly subjective and must inform action utility, making *personalization* essential during EDT recommendation (Example 1.2). Yet, existing data systems provide limited support for EDT recommendations, leading to adoption of AI chatbots: users are exposed to unvalidated suggestions driven by surface-level patterns or popularity signals, not principled notions of exploration such as data coverage and bias avoidance. While incorrect recommendations in entertainment are inconsequential, they can have catastrophic consequences in high-stakes decision-making. This critical risk around AI safety further motivates the need for principled foundations for EDT recommendation.

Example 1.2 (Need for personalization). Consider two users exploring the dataset of Example 1.1 with the same intent of finding discrepancies. One is an investigative journalist and finds evidence of discrimination across demographic groups (e.g., gender-based `salary` disparities) insightful. The other is a company manager and finds discrepancies in `resource utilization` across `team`, `region`, or `year` insightful. The intent and data remain the same, yet the perceived utility of an action differs significantly between users.

While identifying a single action is already challenging, the problem is exacerbated when users must identify a coherent *set* or *sequence* of actions in a *multi-task* setting, with dependencies across tasks, such as data discovery, followed by wrangling, and concluding with summarization. The complexity of this setting requires new theoretical and algorithmic foundations for unified recommendation across multiple, interdependent EDTs (Example 1.3).

Example 1.3 (Unified recommendation in a multi-task setting). During data discovery, a recommender suggests a temperature table from a data lake. During wrangling, a second recommender suggests forward-fill imputation for missing temperature readings based on the time-series semantics inferred during discovery. At the summarization step, a third recommender avoids trends over temperature, recognizing its reduced reliability due to imputation.

Our vision. We envision EDT recommendation as a core capability of data systems, satisfying three key desiderata:

(1) *high utility*, by ensuring relevant, semantically valid, personalized, context-aware, and adaptive recommendations;
(2) *practical efficiency*, by managing the prohibitive computational cost of enumerating a combinatorial number of candidate actions and their materialization; and
(3) *comprehensive* support across single-task single-action, single-task multi-action, and multi-task multi-action settings.

EDT recommendation requires new foundations. *Why are existing solutions in data systems or recommendation systems insufficient?* EDT recommendation systems lie at the intersection of traditional (non-exploratory) data systems and content recommendation systems [1, 6, 34, 35, 37], inheriting challenges from both (Table 1). Traditional data systems typically assume precise specifications and lack support for personalization. Recommendation in data systems mostly focused on bespoke solutions for individual tasks, such as visualization [85] and exploration [42], but they do not generalize to other EDTs or support multiple recommendations.

While personalized recommendation systems have seen significant advances in content domains—e.g., documents in search engines [1, 6], media in streaming platforms [34] and social media [37], and products in e-commerce [35]—their adaptation to EDTs poses two computational challenges. *First*, content recommendation systems typically assume that the candidate space is pre-enumerated, such as a catalog of movies. In contrast, EDTs often involve combinatorially large, programmatically generated spaces of possible *actions* that are not stored explicitly (Example 1.1). *Second,* they assume that all candidates (or their embeddings) are fully observable and can be efficiently scored against a user-preference function. In contrast, assessing the relevance of EDT actions requires (i) costly materialization of candidate actions over the specific data instance (e.g., executing a query over a database) and then (ii) measuring the relevance of the materialized results (e.g., output of a query). In essence, content recommendation systems suggest the data itself, whereas EDTs need recommending *actions*—programs that are applied to data.

Requirements & properties	Traditional Data Systems	Content Rec. Systems	**EDT Rec. Systems**
User-oriented: personalized & adaptive		✓	✓
Subjective goal & vague success metric		✓	✓
Context matters		✓	✓
Novelty/serendipity matters		✓	✓
Semantic understanding required		✓	✓
Interactive speed required		✓	✓
Combinatorial candidate space	✓		✓
Dependency among candidates	✓		✓
Search over sets of candidates	✓		✓
Utility unknown before materialization	✓		✓
High materialization cost	✓		✓

Table 1: EDT recommendation inherits challenges from traditional data systems and content recommendation systems.

LLMs and AI agents: new enablers for EDT recommendation. The recent development of Large Language Models (LLMs) and AI agents makes our vision timely and the unique challenges in EDT recommendation tractable. Specifically, LLMs and AI agents enable development of effective techniques in ways that were not previously possible: (1) LLMs can provide semantic understanding of data to estimate action utility from structural and domain-specific semantics, enabling effective pruning of candidate-action space and materialization cost reduction; and (2) AI agents can track inter-task dependencies and fetch external information, which enables coordinated recommendation across tasks.

However, these capabilities raise a fundamental question: *Can EDT recommendation be solved solely using LLMs and AI agents?* Despite their semantic reasoning and planning capabilities, these systems primarily *generate* plausible but unvalidated actions [17]. However, EDT recommendation is fundamentally a *search and optimization problem, not a generation problem.* Even AI agents remain bottlenecked by the cost of search and validation [44, 81]. Thus, EDT recommendation requires new foundations, with LLMs and agents as enablers, not complete solutions.

AI agents as a new class of EDT users. AI agents are also becoming primary *users* of data systems. Without guidance, an agent brute-forces the action space by repeatedly generating, executing, and inspecting candidate actions. Instead, the agent can call an EDT recommender with its intent and context, and receive validated actions ranked by estimated utility, reducing the actions the agent executes and improving coverage of the action space.

Organization. The rest of the paper is organized as follows. Section 2 presents a categorization of EDT recommendation systems along two axes—recommendation modality and task setting. Section 3

describes the core components of EDT recommendation systems. Section 4 outlines a research agenda based on the key challenges to building theoretical foundations and ensuring the practical requirements of EDT recommendation systems. Section 5 presents an evaluation framework. Section 6 positions our vision with respect to prior work. Section 7 provides concluding remarks.

2 EDT RECOMMENDATION SYSTEM VARIANTS

In this section, we informally formulate the problem of EDT action recommendation and categorize EDT recommendation systems along two axes—recommendation modality and task setting.

Problem 1 (EDT Action Recommendation). *For a specific EDT, given a user, a data instance, constraints, and context, the EDT action recommendation problem is to identify the best action(s) that maximize progress toward the user's (implicit) exploratory goals.*

We envision the EDT recommender as a middleware layer in data systems, between front ends and the database engine. It reuses the query optimizer (statistics, cost estimates, and materialized views) to estimate and validate actions cheaply, and leverages a semantic layer (e.g., LLM-derived semantics) to assess action utility.

Figure 2 shows our categorization of EDT recommendation along two axes: task settings—*single-task*, where recommendations support one EDT, and *multi-task*, where recommendations span multiple EDTs—and recommendation modalities: *single-action*, where top-k actions are recommended, such as insight discovery [21] and visualization [85]; *bundle*, where a set of complementary actions is recommended together, such as data summarization [17] and discovery [70]; and *workflow*, where a sequence of actions is recommended, such as data wrangling [18] and debugging [71]. This categorization yields four variants of EDT recommendation systems.

Top-k recommendation for single task. The simplest setting for EDT recommendation is single-action recommendation within a single-task setting, where users execute one action at a time and the outcome of each action informs next exploration steps. Examples include insight discovery, visualization, data profiling, dataset search, hypothesis exploration, and anomaly discovery. For these tasks, the recommendation system should rank candidate actions by their utility to the user and return the *top-k* actions, allowing users to choose among multiple promising next actions. Notably, solutions for this setting provide the foundation for more complex settings that require recommending multiple actions or spanning multiple EDTs.

Despite their diverse objectives, recommendations across EDTs share a set of fundamental requirements (Table 1, §4). Thus, instead of task-specific bespoke recommendation systems, as exemplified by existing systems in Table 4, a *generalizable framework* should be developed for top-k EDT action recommendation that can be instantiated for any single EDT. The framework should factor out shared components across EDTs, such as search-space reduction and fast candidate-action materialization, to ensure their reuse. It should require only two task-specific components: (i) a language of candidate actions, defined by its vocabulary, grammar, and operational semantics on the data (how to execute an action over a dataset), and (ii) a utility notion to score the materialized results.

Bundle recommendation for single task. Many EDTs involve a *set* of actions [93] that must jointly optimize a global objective—such as maximizing data coverage during data exploration [42]—or

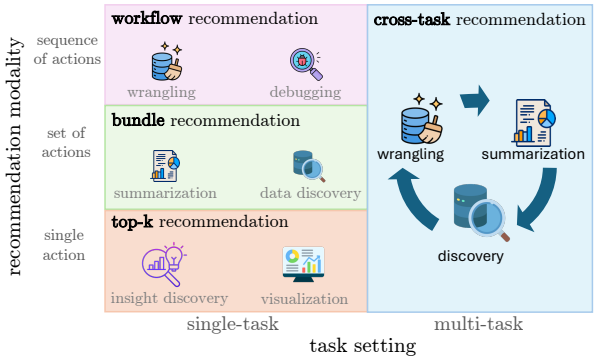


Figure 2: EDT recommendation spans two task settings and three recommendation modalities.

satisfy global constraints—such as ensuring diversity among data summaries [17]. While general constrained set discovery with a global objective is NP-hard, prior solutions for bundle recommendation in data-centric tasks [8, 11, 17, 29, 41, 42, 48, 61, 67, 87] exploit properties of the constraints or objectives, such as submodularity, to devise practically efficient approximate solutions. However, these solutions are often task-specific and do not generalize across EDTs.

Unlike bundle recommendation over standalone items [49], EDT actions require materialization over specific data instances. For example, consider the global constraint of ensuring diversity by requiring the distance between every pair of recommended items to exceed a threshold. The distance between two standalone items c_1 and c_2 can simply be measured as $dist(E(c_1), E(c_2))$, where E maps each item to an embedding space. In contrast, given a database D, the distance between two actions a_1 and a_2 must be computed over their materialized results, $dist(E(a_1(D)), E(a_2(D)))$, where each action a_i is parameterized by D. As such, bundle recommendation for EDTs must strategically *push down* global objectives and constraints into the recommendation process, in the same spirit as predicate pushdown [39, 53, 79, 83, 90] in query optimization.

Workflow recommendation for single task. For EDTs that require a sequence of interdependent actions, decisions made early in the workflow constrain later choices, which makes workflows fundamentally different from unordered bundles. Finding the highest utility workflow for recommendation is a sequence optimization problem, similar to planning, scheduling, and job sequencing. However, since each EDT action transforms the data, subsequent actions operate on the resulting intermediate data. For example, during wrangling, recommending (i) conversion of risk probability to a discrete ordinal scale of 1–5 (for categorical analysis), followed by recommending (ii) missing value imputation using the attribute mean, introduces incompatible values w.r.t. the ordinal scale (e.g., 3.36). This calls for targeted techniques, such as leveraging semantics of actions, to ensure compatibility within the recommended workflow.

Unified recommendations across tasks. A key characteristic of EDTs is their interconnected and interleaved nature in a multi-task setting. Exploratory data analysis may begin with discovery to identify relevant data, proceed through wrangling to clean and reshape the data, and conclude with summarization to extract insights. A one-shot workflow fails in this setting because the process is iterative and actions are mutually dependent. For instance, insights from summarization often trigger further discovery, and newly

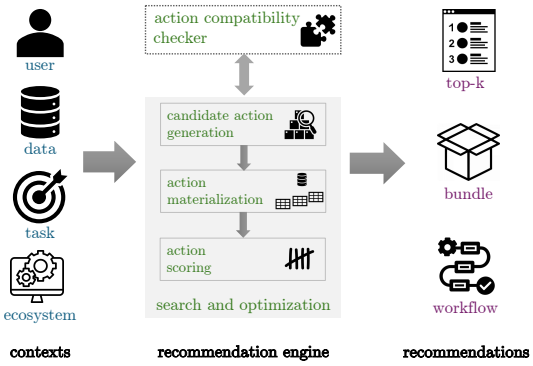


Figure 3: Core components of an EDT recommendation system.

discovered data may require additional wrangling. Recommending for each EDT in isolation fails to account for how recommendations in one task influence recommendations in another, calling for coordination across recommenders (Example 1.3).

3 EDT RECOMMENDATION COMPONENTS

EDT recommendation can be approached through three paradigms. The first is *generation-based*, where generative AI directly constructs a plausible action without considering all possible alternatives. However, such recommendations are unvalidated, providing no guarantee that the recommended action has high utility w.r.t. the data instance. The second is *prediction and ranking based*, prevalent in traditional recommendation systems, where an ML model assigns utility scores to candidate EDT actions based on materialized results. This approach requires candidate actions to be materialized beforehand over the data, and it also requires training data, which is often unavailable in practice. The third paradigm models EDT recommendation as a *search and optimization* problem that leverages the structure of the problem and properties of the utility model and constraints: only promising candidate regions are explored, the utility of materialized results can be approximated, and efficient approximate search algorithms can be devised. We argue that this search-and-optimization paradigm is well suited to EDT recommendation because it directly addresses the combinatorial action space and costly materialization issues inherent to EDTs.

Overview. Figure 3 shows the core components of a generalizable EDT recommendation system within this paradigm. Central to EDT recommendation across all variants are personalization and context-awareness, requiring the system to model user, data, task, and ecosystem *contexts* that serve as input to the *recommendation engine*. The recommendation engine consists of a *search and optimization* module, which efficiently explores the combinatorial action space while reducing the materialization cost required to score candidates, and an *action compatibility checker*, which enforces global constraints and ensures compatibility among recommended actions. The output of the recommendation engine is either *top-k* actions, a *bundle* of actions, or a *workflow* (sequence of actions).

3.1 Contexts

Four sources of context are relevant for EDT action recommendation: (i) **user**: intent, preferences, historical behavior, and feedback; (ii) **task**: the primary goal (e.g., summarization), secondary goals

	Task	Representative work	Contexts			
			user	data semantics	task	ecosystem
Established	Data exploration	Smart Drill-Down [42]	×	×	×	×
	Data preparation	Auto-Suggest [89]	×	×	✓	×
	Data summarization	SAGE [17]	✓	✓	×	×
	Data visualization	SeeDB [85]	×	×	×	×
		ShiftScope [73]	✓	×	✓	×
Emerging	Cherry-picked generalizations	OREO [55]	×	×	×	×
	Explain data change	Explain Da-V [80]	×	×	×	×
	Explain outliers in aggregates	CAPE [59]	×	×	×	×
	Discover entity-enhancing rules	Fan et al. [29]	✓	×	×	×

Table 4: Examples of EDTs and representative work.

(e.g., interpretability), and constraints (e.g., diversity); (iii) **data**: semantics, metadata, and profiles; and (iv) **ecosystem**: workflow, tools in use, and search and communication history. Table 4 shows examples of established and emerging EDTs along with representative works, which provide limited support for these contexts.

Example 3.1 (Leveraging context for unified recommendations). A data scientist begins by cleaning sensor data, where the recommendation system suggests forward-filling imputation for missing temperature readings, leveraging its time-series nature (data context). During feature selection, the system avoids recommending actions that rely on temperature, recognizing its reduced reliability due to imputation (workflow context). During visualization, it suggests binning temperature into wide intervals to reduce noise from imputation (task context) and recommends bar plots, drawing on user preferences observed in other documents (user context).

As Example 3.1 highlights, user context, such as response to earlier recommendations, enables *personalization* and *adaptivity*. Task context provides guidance to model action utility and properties that can guide search and optimization, such as prioritizing task-relevant actions. Data or domain context, which can be obtained from LLMs, provides essential signals for semantic understanding and action utility estimation. Ecosystem context—including recommendations made in other tasks, how prior actions transformed the data, the software environment, search history, and chat or email interactions—enables cross-task coordination.

3.2 Recommendation engine

3.2.1 Search & optimization. This module is responsible for efficiently searching for high-utility EDT actions, which requires managing the large and expensive-to-materialize candidate action space. It generates promising candidates by prioritizing those that align with the user's preferences and task requirements. It further reduces unnecessary materialization by pruning irrelevant, semantically invalid, or low-potential candidates using available context.

For bundle and workflow recommendation, this module searches over combinations and sequences of actions while enforcing global constraints, such as diversity and consistency. It pushes properties of these constraints into candidate generation and materialization, pruning unpromising candidates that cannot contribute to a high-quality bundle or valid workflow. Full materialization is only pursued for the most promising candidates, striking a balance between maintaining high recommendation quality and ensuring low recommendation latency.

3.2.2 Action compatibility checker. This module ensures that recommended actions are coherent and compatible, especially for

workflows, where each action transforms the data and affects subsequent actions that operate on the transformed data. It tracks order dependencies among actions and eliminates candidates that yield incompatible sequences. Further, unified recommendation requires coordination among task-specific recommenders, tracking inter-task dependencies, and obtaining relevant external information.

4 RESEARCH AGENDA

EDT recommendation systems combine requirements from traditional data systems and recommendation systems (Table 1), requiring personalization, adaptation, semantic understanding, and interactive response times. However, the combinatorial action space and the unknown utility of actions until materialization on data pose efficiency challenges. Below, we outline ten research directions toward realizing our vision of EDT recommendation systems.

Learn user preferences over action space. Personalization requires modeling user preferences and intent, which are often implicit and must be learned from interactions [47, 69]. The combinatorial action space makes learning a complete user–action relevance matrix infeasible. However, EDT actions are compositions of a much smaller set of *primitives*, including operations (e.g., `FILTER` or `SPLIT` for wrangling) and their parameters, such as data attributes, aggregate functions, and constants. Thus, a promising direction is to learn user relevance models over these primitives and compose these learned preferences to estimate action relevance.

Model rich data context. Prior work [20, 21, 38, 85] primarily used statistics and metadata to model data context. However, data semantics provide richer signals, which can flag invalid actions (e.g., sum of ZIP codes) and distinguish expected from surprising insights (Example 4.1). LLMs can be utilized to infer such *semantic context* by estimating the probabilities of domain-specific hypotheses.

Example 4.1. Outliers can signal valuable insights—e.g., `PhD` holders in `IT` earning $140K vs. a general average of $60K. However, not all outliers are insightful; some, like this, are expected. In contrast, high blood pressure in kids indicates a genuine insight.

Search space reduction and search acceleration. Possible directions to address the combinatorial search space include: (1) restrict candidate generation to operations and parameters relevant to the user, data, and task contexts, e.g., prioritize user-preferred operations and omit cryptic attributes like `job_code` when interpretability is desired; (2) prune unpromising candidates based on action semantics, such as avoiding "drop rows with any missing value" when 90% of values are missing; (3) bias candidate generation by pushing down global constraints, e.g., avoid similar actions when diversity is desired; and (4) cluster actions across multiple dimensions, such as user preferences, action semantics, and data impact, to focus search on relevant clusters and promote bundle diversity by discouraging multiple selections from the same cluster.

Action materialization cost reduction. Beyond established techniques—such as approximate query processing [63], sampling [5], distributed computing [19], and lazy materialization [91]—LLMs could estimate action utility from action semantics, data statistics, and context. For example, an LLM could estimate filter result size from its condition and data statistics, avoiding materialization. Other directions include anytime materialization [66] that terminates early for unpromising actions and reinforcement learning to predict data-specific utility without materialization. For workflows, materialization could also terminate as soon as incompatibility w.r.t. actions selected earlier is detected. E.g., if earlier actions rely on a functional dependency, a candidate action can be discarded once it modifies the data in a way that violates that dependency.

Decompose global objectives for algorithmic benefits. Recommending a set of actions that maximize a global objective is generally NP-hard [74]. Certain decomposability properties of global objectives, such as submodularity and monotonicity, admit efficient greedy solutions with approximation guarantees. Thus, a promising direction is to develop a taxonomy of decomposable global objectives across EDTs, characterize their algorithmic properties, and identify approximation algorithms that generalize across EDTs.

Decompose global constraints for computational benefits. Learning "action embeddings" could enable decomposition of concepts such as distance, used in global constraints like diversity, to accelerate computation (Example 4.2). Action embeddings should capture three aspects: (1) *structural embedding*, representing action-defining elements such as selection predicates or operations like `fillna`; (2) *semantic embedding*, capturing what the action does, such as compression, addition, or aggregation; and (3) *impact embedding*, representing the action's effect on the data.

Example 4.2 (Decomposing distance). "Distance" can be decomposed into 3 components: (D1-cheapest) structural disjointness of actions, (D2-moderate) data-agnostic semantic difference of actions, and (D3-costliest) divergence in outcomes when actions are applied to data. Under diversity constraints, an action can be skipped if it fails the D1 check w.r.t. the already selected actions.

Model action compatibility. Action compatibility can be modeled as a weighted directed graph, but constructing it over the combinatorial action space is infeasible. A promising direction is building a task-specific *compatibility graph* over action clusters. If no edge connects clusters A and B, no action in A can precede one in B (e.g., lossy transformations cannot precede imputation).

Adaptive recommendation. Adaptive recommendation [57] can incorporate user feedback by modeling the semantics of user interactions with recommended actions. For instance, modifying a filter's threshold signals relevance to the operation but not its parameters. A key challenge is balancing recent feedback with historical behavior. A direction is to learn *temporal user preference embeddings* from edits, refinements, and rejections, capturing short- and long-term preferences through learnable *temporal decay*.

Multi-agent collaboration for cross-task coordination. Agentic approaches enable agents to acquire relevant external information and coordinate across tasks [36, 92]. Multi-agent collaboration [22, 82] could serve as an *outer orchestration layer*, with per-task recommenders providing guardrails through (i) a *context-request* interface for missing context and (ii) a *conflict-report* interface to help resolve cross-task incompatibilities.

Agent-facing recommendation. Serving agents as users raises new challenges. First, the tool should expose machine-readable actions with utility estimates, and accept agent-specified budgets and constraints. Second, agents issue frequent bursts of requests, which calls for sharing computation across requests. Third, agent feedback is abundant but noisy, which requires additional calibration.

5 EVALUATION FRAMEWORK

Evaluating EDT recommendation systems is challenging due to their user-centered nature and the lack of benchmark datasets in this space, which necessitates a multi-pronged evaluation strategy.

Evaluation axes. EDT recommendation systems should be evaluated along six axes using established and new metrics.

Generalizability. Generalizability across tasks and domains should be assessed by evaluating across diverse EDTs and data domains.

Scalability. Scalability should be evaluated under varying data complexity in #tuples & #attributes and candidate action space sizes.

Efficiency. Practical efficiency and fitness for interactive use should be assessed via recommendation latency and reduction of search space and materialization cost.

Validity. Syntactic validity should be measured by error-free execution of the recommended actions. Semantic validity should be measured using (i) precision@k, recall@k, and Mean Reciprocal Rank (MRR) for top-k, (ii) utility and degree of constraint satisfaction for bundles, and (iii) utility and inter-action coherence for workflows. For the multi-task setting, evaluation should also consider success rate in achieving the end goal.

User alignment. Since EDT recommendation targets human users, it must be validated via user studies, including controlled experiments, interviews, and contextual observation using think-aloud protocols. Metrics should include increase in user satisfaction over EDT iterations to measure *adaptivity* and across sessions to measure *personalization*. Likert-scale questions such as "How frequently did you find the recommended actions relevant?" can quantify user satisfaction, while #iterations and time to task completion can measure the impact of recommendations on user efficiency. For agents, the impact should be measured by comparing agents that leverage the recommender against agents that do not, in terms of execution cost, token usage, and task success rate.

Robustness and parameter sensitivity. Robustness should be evaluated under noise, data skew, intent ambiguity, and task complexity. Sensitivity to system parameters, optimization techniques, and materialization strategies should be analyzed via ablation studies.

Human-validated utility models for evaluation at scale. Validating recommendations by obtaining accept/reject feedback via user studies is prohibitively expensive and cognitively demanding: without assessing the utility of the alternatives, users cannot correctly judge recommendation fitness. A solution is to use *human-validated utility models* that capture real-user preferences and objectively quantify recommendation quality [17]. This involves (i) devising utility models covering various EDT-specific criteria such as relevance, novelty, diversity, coverage, etc.; (ii) a small-scale user study over several use cases to identify the best model; and (iii) using the validated model to evaluate recommendations at scale.

Benchmark datasets. Benchmark datasets should be created in three ways: (i) workflow extraction from existing sources such as notebooks [86]; (ii) collection of real workflows from user studies over prototypes; and (iii) brute-force search to identify ground truths: actions that should be recommended based on user-validated utility models. For personalization and adaptivity evaluation, a direction is to create a benchmark with a list of intents (Example 1.2), and obtain ground-truth workflows from domain experts.

6 RELATED WORK

Existing suggestion systems target specific tasks such as data visualization [7, 20, 25, 40, 45, 52, 65, 73, 85], exploration [12, 23, 24, 26, 33, 56, 58, 62, 75], wrangling [18, 43, 64], analysis [51, 60], explanation [8, 9], summarization [16, 17], etc. Some focus on suggesting the mechanism (e.g., queries) [14, 26, 28, 30–32, 46, 50], while others focus on the results (e.g., tuples). Some consider user profiles, while others leverage prior query logs. RecDB [76, 77] supports personalized data suggestion but does not suggest the underlying query. ATENA [26] generates an entire EDA session but without keeping a human in the loop. Suggestion engines have also been explored for other data-management tasks such as database tuning and indexing [2, 54, 78, 84], clustering [10], time-series generation [3], identifying valuable attributes in OLAP [15], view generation [27], and data-science pipeline construction [68]. However, none of these works offers a framework that generalizes across EDTs.

Some approaches use machine learning, such as reinforcement learning [73] or convolutional neural networks [40], which assume small candidate space. Most require materializing all candidates to evaluate their relevance [65] and support only top-k suggestions [20, 25, 40, 65, 85]. For suggesting spreadsheet formulas and data-preparation steps, Auto-Formula [13] and Auto-Suggest [89] use machine learning to predict top-k suggestions. Generally, machine-learning models treat suggesting the next EDT action as a standard prediction problem. A vast literature exists for content recommendation [72], spanning multiple communities, from information retrieval and recommender systems to data mining, artificial intelligence, and machine learning. However, adapting content recommendation systems to EDTs is nontrivial.

In summary, prior works have several limitations: (1) they target a specific task and offer a bespoke solution; (2) they often ignore signals such as the user's workflow and ecosystem context, failing to align with the user intent; and (3) they are history-driven, relying heavily on ML-based approaches akin to traditional recommendation engines, which are ineffective for EDTs.

7 CONCLUSIONS

We presented a vision of recommendation systems for exploratory data tasks where recommendation capability is a first-class citizen of data systems, not an afterthought or a superficial chatbot feature. We characterized EDT recommendation across single-task and multi-task settings and single-action, bundle, and workflow modalities. We provided a design of a generalizable and unified recommendation framework, outlined a research agenda spanning context modeling, search-space and materialization cost reduction, and cross-task coordination, and devised an evaluation framework. Realizing this vision requires addressing the unique challenges of EDT recommendation by integrating techniques from data management, recommendation systems, and constrained optimization, with capabilities of large language models and AI agents. Our vision is to make action recommendations for EDTs more trustworthy, effective, and accessible, democratizing both existing and emerging tasks, for human users and AI agents, which can ultimately empower millions of analysts, scientists, engineers, journalists, and policymakers to perform complex exploratory data tasks more accurately and efficiently in diverse real-world settings.

REFERENCES

[1] Himan Abdollahpouri, Robin Burke, and Bamshad Mobasher. 2017. Controlling popularity bias in learning-to-rank recommendation. In *Proceedings of the eleventh ACM conference on recommender systems*. 42–46.

[2] Parinaz Ameri. 2016. On a self-tuning index recommendation approach for databases. In *2016 IEEE 32nd International Conference on Data Engineering Workshops (ICDEW)*. 201–205.

[3] Yihao Ang, Yifan Bao, Qiang Huang, Anthony K. H. Tung, and Zhiyong Huang. 2024. TSGAssist: An Interactive Assistant Harnessing LLMs and RAG for Time Series Generation Recommendations and Benchmarking. *Proc. VLDB Endow.* 17, 12 (2024), 4309–4312.

[4] Marcelo Arenas, Enrico Franconi, Janik Hammerer, Olaf Hartig, Katja Hose, Laura Koesten, George Konstantinidis, Leonid Libkin, Wim Martens, Yuya Sasaki, Stefanie Scherzinger, Katherine Thornton, and Hsiang-Yun Wu. 2025. Exploring Exploratory Querying. *Proc. VLDB Endow.* 18, 13 (Sept. 2025), 5731–5739.

[5] Brian Babcock, Surajit Chaudhuri, and Gautam Das. 2003. Dynamic sample selection for approximate query processing. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*. 539–550.

[6] Ricardo Baeza-Yates, Carlos Hurtado, and Marcelo Mendoza. 2004. Query recommendation using query logs in search engines. In *International conference on extending database technology*. Springer, 588–596.

[7] Eduardo M. Barbosa, Mirella M. Moro, Giseli Rabello Lopes, and José Palazzo Moreira de Oliveira. 2012. VRRC: web based tool for visualization and recommendation on co-authorship network (abstract only). In *SIGMOD*. ACM, 865.

[8] Tal Blau, Brit Youngmann, Anna Fariha, and Yuval Moskovitch. 2026. Causal Explanations for Disparate Trends: Where and Why? *Proc. ACM Manag. Data* 4, 1 (2026), 17:1–17:26.

[9] Tal Blau, Brit Youngmann, Anna Fariha, and Yuval Moskovitch. 2026. ExDis: Causal Explanations for Disparate Trends. In *Companion of the International Conference on Management of Data, SIGMOD Companion 2026, 31 May 2026 - 5 June 2026*. ACM, 14–17.

[10] Suratna Budalakoti, Mohamed Ziauddin, Andrew Witkowski, You Jung Kim, Ramarajan Krishnamachari, and Alan Wood. 2024. Automated Clustering Recommendation With Database Zone Maps. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, June 9-15, 2024*. ACM, 68–79.

[11] Felix S. Campbell, Alon Silberstein, Julia Stoyanovich, and Yuval Moskovitch. 2024. Query Refinement for Diverse Top-k Selection. *Proc. ACM Manag. Data* 2, 3, Article 166 (May 2024), 27 pages.

[12] Gloria Chatzopoulou, Magdalini Eirinaki, and Neoklis Polyzotis. 2009. Query Recommendations for Interactive Database Exploration. In *Proceedings of the 21st International Conference on Scientific and Statistical Database Management (SSDBM 2009)*. Springer-Verlag, 3–18.

[13] Sibei Chen, Yeye He, Weiwei Cui, Ju Fan, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. 2024. Auto-Formula: Recommend Formulas in Spreadsheets using Contrastive Learning for Table Representations. *Proc. ACM Manag. Data* 2, 3 (2024), 122.

[14] Wanyu Chen, Fei Cai, Honghui Chen, and Maarten de Rijke. 2017. Personalized Query Suggestion Diversification. In *SIGIR*, Noriko Kando, Tetsuya Sakai, Hideo Joho, Hang Li, Arjen P. de Vries, and Ryen W. White (Eds.). ACM, 817–820.

[15] Xingguang Chen, Fangyuan Zhang, Jinchao Huang, and Sibo Wang. 2023. Efficient Approximation Framework for Attribute Recommendation. *Proc. ACM Manag. Data* 1, 4 (2023), 239:1–239:26.

[16] Whanhee Cho and Anna Fariha. 2024. UTOPIA: Automatic Pivot Table Assistant. *Proc. VLDB Endow.* 17, 12 (2024), 4305–4308.

[17] Whanhee Cho and Anna Fariha. 2026. Data-Semantics-Aware Recommendation of Diverse Pivot Tables. *Proc. ACM Manag. Data* 4, 1 (2026), 23:1–23:28.

[18] Bhavya Chopra, Anna Fariha, Sumit Gulwani, Austin Z. Henley, Daniel Perelman, Mohammad Raza, Sherry Shi, Danny Simmons, and Ashish Tiwari. 2023. CoWrangler: Recommender System for Data-Wrangling Scripts. In *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, June 18-23, 2023*. ACM, 147–150.

[19] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (2008), 107–113.

[20] Çagatay Demiralp, Peter J. Haas, Srinivasan Parthasarathy, and Tejaswini Pedapati. 2017. Foresight: Recommending Visual Insights. *Proc. VLDB Endow.* 10, 12 (2017), 1937–1940.

[21] Rui Ding, Shi Han, Yong Xu, Haidong Zhang, and Dongmei Zhang. 2019. QuickInsights: Quick and Automatic Discovery of Insights from Multi-Dimensional Data. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, June 30 - July 5, 2019*. ACM, 317–332.

[22] Ali Dorri, Salil Subhash Kanhere, and Raja Jurdak. 2018. Multi-Agent Systems: A Survey. *IEEE Access* 6 (2018), 28573–28593.

[23] Marina Drosou and Evaggelia Pitoura. 2011. ReDRIVE: result-driven database exploration through recommendations. In *Proceedings of the 20th ACM International Conference on Information and Knowledge Management (CIKM '11)*. Association for Computing Machinery, 1547–1552.

[24] Marina Drosou and Evaggelia Pitoura. 2013. YmalDB: exploring relational databases via result-driven recommendations. *The VLDB Journal* 22, 6 (Dec. 2013), 849–874.

[25] Humaira Ehsan, Mohamed A. Sharaf, and Panos K. Chrysanthis. 2016. MuVE: Efficient Multi-Objective View Recommendation for Visual Data Exploration. In *32nd IEEE International Conference on Data Engineering, ICDE 2016, May 16-20, 2016*. IEEE Computer Society, 731–742.

[26] Ori Bar El, Tova Milo, and Amit Somech. 2020. Automatically Generating Data Exploration Sessions Using Deep Reinforcement Learning. In *SIGMOD*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 1527–1537.

[27] Amr El-Helw, Ihab F. Ilyas, and Calisto Zuzarte. 2009. StatAdvisor: recommending statistical views. *Proc. VLDB Endow.* 2, 2 (Aug. 2009), 1306–1317.

[28] Ju Fan, Guoliang Li, and Lizhu Zhou. 2011. Interactive SQL query suggestion: Making databases user-friendly. In *2011 IEEE 27th International Conference on Data Engineering*. 351–362.

[29] Wenfei Fan, Ziyan Han, Min Xie, and Guangyi Zhang. 2024. Discovering Top-k Relevant and Diversified Rules. *Proc. ACM Manag. Data* 2, 4 (2024), 195:1–195:28.

[30] Anna Fariha and Alexandra Meliou. 2019. Example-Driven Query Intent Discovery: Abductive Reasoning using Semantic Similarity. *Proc. VLDB Endow.* 12, 11 (2019), 1262–1275.

[31] Matteo Francia, Matteo Golfarelli, Patrick Marcel, Stefano Rizzi, and Panos Vassiliadis. 2023. Suggesting Assess Queries for Interactive Analysis of Multidimensional Data. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2023), 6421–6434.

[32] Arnaud Giacometti, Patrick Marcel, and Elsa Negre. 2009. Recommending Multidimensional Queries. In *Proceedings of the 11th International Conference on Data Warehousing and Knowledge Discovery (DaWaK '09)*. Springer-Verlag, 453–466.

[33] Apostolos Glenis and Georgia Koutrika. 2021. PyExplore: Query Recommendations for Data Exploration without Query Logs. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, 2731–2735.

[34] Carlos Alberto Gomez-Uribe and Neil Hunt. 2016. The Netflix Recommender System: Algorithms, Business Value, and Innovation. *ACM Trans. Manag. Inf. Syst.* 6, 4 (2016), 13:1–13:19.

[35] Mihajlo Grbovic, Vladan Radosavljevic, Nemanja Djuric, Narayan Bhamidipati, Jaikit Savla, Varun Bhagwan, and Doug Sharp. 2015. E-commerce in Your Inbox: Product Recommendations at Scale. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, August 10-13, 2015*. 1809–1818.

[36] Florian Grötschla, Luis Müller, Jan Tönshoff, Mikhail Galkin, and Bryan Perozzi. 2025. AgentsNet: Coordination and Collaborative Reasoning in Multi-Agent LLMs. *arXiv preprint arXiv:2507.08616* (2025).

[37] Pankaj Gupta, Venu Satuluri, Ajeet Grewal, Siva Gurumurthy, Volodymyr Zhabiuk, Quannan Li, and Jimmy Lin. 2014. Real-time Twitter recommendation: Online motif detection in large dynamic graphs. *Proceedings of the VLDB Endowment* 7, 13 (2014), 1379–1380.

[38] Camille Harris, Ryan A. Rossi, Sana Malik, Jane Hoffswell, Fan Du, Tak Yeon Lee, Eunyee Koh, and Handong Zhao. 2023. SpotLight: Visual Insight Recommendation. In *Companion Proceedings of the ACM Web Conference 2023, WWW 2023, 30 April 2023 - 4 May 2023*. ACM, 19–23.

[39] Joseph M Hellerstein and Michael Stonebraker. 1993. Predicate migration: Optimizing queries with expensive predicates. In *Proceedings of the 1993 ACM SIGMOD international conference on Management of data*. 267–276.

[40] Daomin Ji, Hui Luo, and Zhifeng Bao. 2023. Visualization Recommendation Through Visual Relation Learning and Visual Preference Learning. In *39th IEEE International Conference on Data Engineering, ICDE 2023, April 3-7, 2023*. IEEE, 1860–1873.

[41] Zhongjun Jin, Yeye He, and Surajit Chaudhuri. 2020. Auto-Transform: Learning-to-Transform by Patterns. *Proc. VLDB Endow.* 13, 11 (2020), 2368–2381.

[42] Manas Joglekar, Hector Garcia-Molina, and Aditya G. Parameswaran. 2016. Interactive data exploration with smart drill-down. In *32nd IEEE International Conference on Data Engineering, ICDE 2016, May 16-20, 2016*. IEEE Computer Society, 906–917.

[43] Anirudh Kamath, Dustin Maas, Jacobus van der Merwe, and Anna Fariha. 2026. WN-Wrangle: Wireless Network Data Wrangling Assistant. *CoRR* abs/2603.21310 (2026). arXiv:2603.21310

[44] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. 2024. Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. In *Forty-first International Conference on Machine Learning, ICML 2024, July 21-27, 2024 (Proceedings of Machine Learning Research)*, Vol. 235. PMLR / OpenReview.net, 22895–22907.

[45] Alicia Key, Bill Howe, Daniel Perry, and Cecilia R. Aragon. 2012. VizDeck: self-organizing dashboards for visual analytics. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, May 20-24, 2012*. ACM, 681–684.

[46] Nodira Khoussainova, YongChul Kwon, Magdalena Balazinska, and Dan Suciu. 2010. SnipSuggest: context-aware autocompletion for SQL. *Proc. VLDB Endow.* 4, 1 (Oct. 2010), 22–33.

[47] Jonathan Koren, Yi Zhang, and Xue Liu. 2008. Personalized interactive faceted search. In *Proceedings of the 17th international conference on World Wide Web*. 477–486.

[48] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: Interactive Data Cleaning For Statistical Modeling. *Proc. VLDB Endow.* 9, 12 (2016), 948–959.

[49] Matevž Kunaver and Tomaž Požrl. 2017. Diversity in recommender systems - A survey. *Knowl. Based Syst.* 123 (2017), 154–162.

[50] Eugenie Yujing Lai, Zainab Zolaktaf, Mostafa Milani, Omar AlOmeir, Jianhao Cao, and Rachel Pottinger. 2023. Workload-Aware Query Recommendation Using Deep Learning. In *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, March 28-31, 2023*. OpenProceedings.org, 53–65.

[51] Doris Jung Lin Lee, Abdul Quamar, Eser Kandogan, and Fatma Özcan. 2021. Boomerang: Proactive Insight-Based Recommendations for Guiding Conversational Data Analysis. In *SIGMOD '21: International Conference on Management of Data, June 20-25, 2021*. ACM, 2750–2754.

[52] Doris Jung Lin Lee, Dixin Tang, Kunal Agarwal, Thyne Boonmark, Caitlyn Chen, Jake Kang, Ujjaini Mukhopadhyay, Jerry Song, Micah Yong, Marti A. Hearst, and Aditya G. Parameswaran. 2021. Lux: Always-on Visualization Recommendations for Exploratory Dataframe Workflows. *Proc. VLDB Endow.* 15, 3 (2021), 727–738.

[53] Alon Y. Levy, Inderpal Singh Mumick, and Yehoshua Sagiv. 1994. Query Optimization by Predicate Move-Around. In *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994*. 96–107.

[54] Guoliang Li, Xuanhe Zhou, Shifu Li, and Bo Gao. 2019. QTune: a query-aware database tuning system with deep reinforcement learning. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2118–2130.

[55] Yin Lin, Brit Youngmann, Yuval Moskovitch, H. V. Jagadish, and Tova Milo. 2021. On Detecting Cherry-picked Generalizations. *Proc. VLDB Endow.* 15, 1 (2021), 59–71.

[56] Tavor Lipman, Tova Milo, Amit Somech, Tomer Wolfson, and Oz Zafar. 2025. LINX: A Language Driven Generative System for Goal-Oriented Automated Data Exploration. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, March 25-28, 2025*, Alkis Simitsis, Bettina Kemme, Anna Queralt, Oscar Romero, and Petar Jovanovic (Eds.). OpenProceedings.org, 270–283.

[57] Guilherme Medeiros Machado, Vinicius Maran, Gabriel Machado Lunardi, Leandro Krug Wives, and José Palazzo Moreira de Oliveira. 2021. AwARE: a framework for adaptive recommendation of educational resources. *Computing* 103, 4 (2021), 675–705.

[58] Antonis Mandamadiotis, Georgia Koutrika, Stavroula Eleftherakis, Apostolos Glenis, Dimitrios Skoutas, and Yannis Stavrakas. 2021. DatAgent: The Imminent Age of Intelligent Data Assistants. *Proc. VLDB Endow.* 14, 12 (2021), 2815–2818. http://www.vldb.org/pvldb/vol14/p2815-mandamadiotis.pdf

[59] Zhengjie Miao, Qitian Zeng, Boris Glavic, and Sudeepa Roy. 2019. Going Beyond Provenance: Explaining Query Answers with Pattern-based Counterbalances. In *SIGMOD*. ACM, 485–502.

[60] Tova Milo and Amit Somech. 2016. REACT: Context-Sensitive Recommendations for Data Analysis. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, June 26 - July 01, 2016*. ACM, 2137–2140.

[61] Zafeiria Moumoulidou, Andrew McGregor, and Alexandra Meliou. 2021. Diverse Data Selection under Fairness Constraints. In *24th International Conference on Database Theory, ICDT 2021, March 23-26, 2021 (LIPIcs)*, Vol. 186. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 13:1–13:25.

[62] Quoc Viet Hung Nguyen, Kai Zheng, Matthias Weidlich, Bolong Zheng, Hongzhi Yin, Thanh Tam Nguyen, and Bela Stantic. 2018. What-If Analysis with Conflicting Goals: Recommending Data Ranges for Exploration. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. 89–100.

[63] Yongjoo Park, Barzan Mozafari, Joseph Sorenson, and Junhao Wang. 2018. VerdictDB: Universalizing approximate query processing. In *Proceedings of the 2018 International Conference on Management of Data*. 1461–1476.

[64] Tomas Petricek, Gerrit J. J. van den Burg, Alfredo Nazábal, Taha Ceritli, Ernesto Jiménez-Ruiz, and Christopher K. I. Williams. 2023. AI Assistants: A Framework for Semi-Automated Data Wrangling. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (2023), 9295–9306.

[65] Xin Qian, Ryan A. Rossi, Fan Du, Sungchul Kim, Eunyee Koh, Sana Malik, Tak Yeon Lee, and Joel Chan. 2021. Learning to Recommend Visualizations from Data. In *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, August 14-18, 2021*. ACM, 1359–1369.

[66] Vijayshankar Raman and Joseph M Hellerstein. 2002. Partial results for online query processing. In *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*. 275–286.

[67] Sayan Ranu, Minh Hoang, and Ambuj Singh. 2014. Answering top-k representative queries on graph databases. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 1163–1174.

[68] Sergey Redyuk, Zoi Kaoudi, Sebastian Schelter, and Volker Markl. 2022. DORIAN in action: assisted design of data science pipelines. *Proc. VLDB Endow.* 15, 12 (Aug. 2022), 3714–3717.

[69] Paul Resnick and Hal R Varian. 1997. Recommender systems. *Commun. ACM* 40, 3 (1997), 56–58.

[70] El Kindi Rezig, Anshul Bhandari, Anna Fariha, Benjamin Price, Allan Vanterpool, Vijay Gadepally, and Michael Stonebraker. 2021. DICE: Data Discovery by Example. *Proc. VLDB Endow.* 14, 12 (2021), 2819–2822.

[71] El Kindi Rezig, Lei Cao, Giovanni Simonini, Maxime Schoemans, Samuel Madden, Nan Tang, Mourad Ouzzani, and Michael Stonebraker. 2020. Dagger: A Data (not code) Debugger. In *10th Conference on Innovative Data Systems Research, CIDR 2020, January 12-15, 2020, Online Proceedings*. www.cidrdb.org.

[72] RUCAIBox. 2025. Awesome-RSPapers: Recommender System Papers. https://github.com/RUCAIBox/Awesome-RSPapers. Accessed: 2025-07-05.

[73] Sanad Saha, Nischal Aryal, Leilani Battle, and Arash Termehchy. 2024. ShiftScope: Adapting Visualization Recommendations to Users' Dynamic Data Focus. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, June 9-15, 2024*. ACM, 536–539.

[74] Shuichi Sakai, Mitsunori Togasaki, and Koichi Yamazaki. 2003. A note on greedy algorithms for the maximum weighted independent set problem. *Discrete Applied Mathematics* 126, 2-3 (2003), 313–322.

[75] Sunita Sarawagi. 2000. User-Adaptive Exploration of Multidimensional Data. In *VLDB 2000, Proceedings of 26th International Conference on Very Large Data Bases, September 10-14, 2000*, Amr El Abbadi, Michael L. Brodie, Sharma Chakravarthy, Umeshwar Dayal, Nabil Kamel, Gunter Schlageter, and Kyu-Young Whang (Eds.). Morgan Kaufmann, 307–316. http://www.vldb.org/conf/2000/P307.pdf

[76] Mohamed Sarwat, James L. Avery, and Mohamed F. Mokbel. 2013. RecDB in Action: Recommendation Made Easy in Relational Databases. *Proc. VLDB Endow.* 6, 12 (2013), 1242–1245.

[77] Mohamed Sarwat, Raha Moraffah, Mohamed F. Mokbel, and James L. Avery. 2017. Database System Support for Personalized Recommendation Applications. In *33rd IEEE International Conference on Data Engineering, ICDE 2017, April 19-22, 2017*. IEEE Computer Society, 1320–1331.

[78] Karl Schnaitter and Neoklis Polyzotis. 2012. Semi-automatic index tuning: keeping DBAs in the loop. *Proc. VLDB Endow.* 5, 5 (Jan. 2012), 478–489.

[79] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.

[80] Roee Shraga and Renée J. Miller. 2023. Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V. *Proc. VLDB Endow.* 16, 6 (2023), 1587–1600.

[81] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2025. On the self-verification limitations of large language models on reasoning and planning tasks. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, April 24-28, 2025*. OpenReview.net.

[82] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-agent collaboration: Harnessing the power of intelligent LLM agents. *arXiv preprint arXiv:2306.03314* (2023).

[83] Jeffrey D. Ullman. 1988–1989. *Principles of Database and Knowledge-Base Systems. Volume 1 and 2*. Computer Science Press.

[84] G. Valentin, M. Zuliani, D.C. Zilio, G. Lohman, and A. Skelley. 2000. DB2 advisor: an optimizer smart enough to recommend its own indexes. In *Proceedings of 16th International Conference on Data Engineering*. 101–110.

[85] Manasi Vartak, Sajjadur Rahman, Samuel Madden, Aditya G. Parameswaran, and Neoklis Polyzotis. 2015. SEEDB: Efficient Data-Driven Visualization Recommendations to Support Visual Analytics. *Proc. VLDB Endow.* 8, 13 (2015), 2182–2193.

[86] Yuqing Wang and Anna Fariha. 2024. Development of Data Evaluation Benchmark for Data Wrangling Recommendation System. *CoRR* abs/2409.10635 (2024). arXiv:2409.10635

[87] Yue Wang, Alexandra Meliou, and Gerome Miklau. 2018. RC-index: diversifying answers to range queries. *Proc. VLDB Endow.* 11, 7 (March 2018), 773–786.

[88] Junjie Xing, Xinyu Wang, and H. V. Jagadish. 2024. Data-Driven Insight Synthesis for Multi-Dimensional Data. *Proc. VLDB Endow.* 17, 5 (2024), 1007–1019.

[89] Cong Yan and Yeye He. 2020. Auto-Suggest: Learning-to-Recommend Data Preparation Steps Using Data Science Notebooks. In *SIGMOD*. ACM, 1539–1554.

[90] Cong Yan, Yin Lin, and Yeye He. 2023. Predicate pushdown for data science pipelines. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–28.

[91] Jingren Zhou, Per-Ake Larson, and Hicham G Elmongui. 2007. Lazy maintenance of materialized views. In *Proceedings of the 33rd international conference on Very large data bases*. 231–242.

[92] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. 2025. MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents. *CoRR* abs/2503.01935 (2025). arXiv:2503.01935

[93] Tao Zhu, Patrick Harrington, Junjun Li, and Lei Tang. 2014. Bundle recommendation in ecommerce. In *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval*. 657–666.